MyCodeSchool

Code review — stefan-academy-vanilla-csharp

Runde de review pe proiectul tău, cea mai nouă sus.

Proiect: mini-ERP de academie, C# consolă, fără framework. Regula fișierului: runda nouă se adaugă **sus**. Rundele vechi rămân dedesubt, condensate.

Runda 12 — 2026-10-14 — sha `c2b024b`

Scop: verificat commit-ul `c2b024b` „code review fixes", adică răspunsul tău la runda 11 (B1–B5 + cele 5 🟡).

Metodă: `dotnet build`, apoi o copie în scratchpad, cu datele resetate înainte de fiecare drum. Am rulat 10 drumuri cu inputul tastat: carte nouă + aceeași carte încă o dată, carte cu nume invalid (adăugare și modificare), înscriere de două ori la Algebra, tastă greșită în meniul de admin, în meniul principal și în Statistici, vârsta `doua`, tasta 0 în Cărți și în Cursuri, login cu un nume care nu există și cu parola greșită. Fiecare 🔴 de mai jos e reprodus.

Poarta de compilare: ✅ **0 erori**, al 8-lea commit verde la rând.

🎉 Ce merge

Ce	Dovada
✅ **B1/r11: cărțile se adaugă din nou**	„Dune Noua" → `Cartea Dune Noua adaugata cu succes`; a doua oară → `e deja in lista`. Verificarea e pusă **înainte** de `BookMapper.ToBook`, exact cum cerea fix-ul
✅ **B5/r11: vârsta `doua`**	`Varsta trebuie sa fie un numar.`, iar meniul de admin rămâne pe ecran
✅ **B4/r11 în 3 din 5 meniuri**	`x` în meniul de admin, în meniul principal și în Statistici → `Ati introdus un caracter nepermis!` și meniul rămâne
✅ **Regula convertită în ambele fișiere**	`Book` (3) și `Course` (6): toate cele 9 `throw` sunt acum `ValidationException`. De data asta n-ai lăsat nicio apariție în urmă
Fără stack trace la înscrierea dublă	`Main` prinde acum `AcademyException` și afișează `MesajPentruUtilizator`
`using`-uri moarte scoase	`Program.cs`

🔴 Critice

B1 — Din Cărți și din Cursuri nu mai poți ieși

`ViewStudent.cs:84` și `:113`

CUM E ACUM: logat ca Ștefan Scarlat, intru în Cursuri și apăs `0` de patru ori:

```
Apasati tasta 0 pentru a va intoarce
Apasati tasta 1 pentru a vedea cursurile
...
Apasati tasta 0 pentru a va intoarce
Apasati tasta 1 pentru a vedea cursurile
...
```

Meniul se reafișează la nesfârșit. Nu te mai poți întoarce la meniul principal și nu poți ieși din aplicație decât cu Ctrl+C. La fel în Cărți.

DE CE: în cele două submeniuri, ramura de ieșire este

```
case 0: break;
```

`break` într-un `switch` iese din `switch`, nu din buclă. Până acum nu se vedea, pentru că ieșirea o făcea condiția: `break` termina `switch`-ul, se ajungea la `while (tasta != 0)`, iar condiția oprea bucla. Ai schimbat condiția în `while (true)`, deci singurul lucru care te scotea afară a dispărut. `break` face în continuare exact ce făcea și înainte, adică aproape nimic.

În celelalte 3 meniuri merge, pentru că acolo scria deja `case 0: return;`. Și partea mea de vină: în runda 11 am scris „ieșirea e doar `case 0: return;` (pe care îl ai deja)". Era adevărat pentru 3 meniuri, nu pentru toate 5. Când schimbi condiția unei bucle, verifici **fiecare** drum prin care bucla se terminase până atunci.

FIX: `case 0: return;` la `:84` și la `:113`.

B2 — O greșeală de tastare la login închide aplicația cu stack trace

`Program.cs:19` + `ViewLogIn.cs:40, :61, :75, :85`

CUM E ACUM: la login scriu un nume care nu există:

```
Numele: Prenumele: Unhandled exception. System.ArgumentException: Userul nu
aceste credentiale nu exista
   at stefan_academy_vanilla_charp.ViewLogIn.Logger() in ViewLogIn.cs:line 40
   at Program.Main() in Program.cs:line 17
```

La fel cu parola greșită la admin (`ViewLogIn.cs:61`). În runda 11 aceleași două greșeli afișau un mesaj curat.

DE CE: ai **înlocuit** `catch (ArgumentException)` cu `catch (AcademyException)`. Dar `ViewLogIn` aruncă tot `ArgumentException`, în 4 locuri. Cele două tipuri nu au nimic în comun: `ArgumentException` vine din biblioteca .NET, `AcademyException` e a ta, și singurul strămoș comun e `Exception`. Așa că acum nimeni nu mai prinde excepțiile de la login. Runtime-ul oprește programul și afișează stack trace-ul.

Runda 11 cerea „al doilea `catch`, pe `AcademyException`". Al doilea înseamnă în plus, nu în locul celui vechi. Un `catch` nou acoperă un tip nou, dar nu le acoperă pe cele vechi.

FIX: `ViewLogIn` primește aceeași regulă ca restul proiectului. Userul care nu există → `NotFoundException("Userul", lastName + " " + firstName)`, parola greșită → `ValidationException("Parola", "gresita")`. După asta, `catch (AcademyException)` din `Main` le prinde pe toate.

B3 — B2 și B3 din runda 11, pe jumătate: nu mai crapă, dar te scot din aplicație

`ViewStudent.cs:199`, `:225`, `:267`

CUM E ACUM: modific `Cartea1` în `Carte!`:

```
Tastati noul nume al cartii: Campul Numele nu e bun:  contine caractere nepermise

[exit=0]
```

Mesajul e bun, dar după el programul s-a terminat. Comanda următoare (`1`, afișare cărți) nu s-a mai executat. La fel la adăugarea lui `Carte!` și la a doua înscriere la Algebra.

DE CE: acum excepția **este** prinsă, dar în locul greșit. Lanțul arată așa:

1. `Book` aruncă `ValidationException` (corect, ai făcut asta);
2. în `ViewStudent`, `catch (DuplicateException)` la `:199` și `catch (NotFoundException)` la `:225` sunt tot înguste, deci excepția trece de ele. `InscriereCurs` nu are niciun `try`;
3. excepția urcă până la `catch (AcademyException)` din `Main`. Acela stă în **afara** buclei de meniu. După ce se execută un `catch`, programul continuă cu linia de **după** el, iar în `Main` după `catch` nu mai urmează nimic.

Locul unde prinzi o excepție hotărăște unde continuă programul. Plasa din `Main` e bună ca ultimă plasă, dar nu poate să te țină în meniu. Ca să rămâi în meniu, excepția trebuie prinsă **sub** bucla meniului, în metoda care a cerut operația.

FIX: cele trei puncte din runda 11, nefăcute: `catch (AcademyException ex)` la `:199` și `:225`, plus `try` / `catch (AcademyException ex)` în jurul lui `CreateEnrolment` la `:267`.

🟡 Importante

- **M1 — `RASPUNSURI.txt` tot nu există.** Sunt patru rânduri de întrebări fără răspuns acum: runda 9, runda 10, fișa Singleton, runda 11. B2 și B3 de mai sus sunt exact întrebarea 3 din runda 11 („un `catch (AcademyException)` prinde un `ArgumentException`?"). Dacă ai fi scris răspunsul, ai fi găsit singur bug-ul.
- **M2 — `Data/jurnal.txt` e tot în git, iar `.gitignore` e neatins** (runda 8, 10, 11). `git rm --cached Data/jurnal.txt` + linia `Data/jurnal.txt` în `.gitignore`.
- **M3 — `TextMapperCuJurnal` primește o cale pe care n-o folosește, și calea e greșită.** Câmpul `path` (`:6`) e setat, dar `Write` (`:29`) își construiește tot singur calea spre `jurnal.txt`. Iar `UserRepository.cs:25` îi dă calea spre `users.txt`. Când legi câmpul de `Write`, jurnalul scrie peste fișierul cu utilizatori, iar la pornirea următoare `FromText` primește linia `User cu jurnal: ...` și aplicația nu mai pornește. Calea vine din `Main`, spre `jurnal.txt`, ca la `NotificatorFisier`.
- **M4 — `ModificareCarte` tot nu confirmă nimic** (a 5-a rundă). Singura linie după o modificare reușită e `NotificatorTest Book updated successfully`.
- **M5 — regula „fără carte dublă" stă doar pe drumul de adăugare.** Am modificat `CarteaUnu` în `Alchimistul` și am ajuns la trei cărți `Alchimistul` (`BookService.cs:44–55` nu verifică). Și adăugarea se ocolește cu un spațiu în față: `"`

`Alchimistul"` trece de verificare, pentru că `BookService.cs:32` caută textul netăiat, iar `Book.cs:34` salvează textul tăiat cu `Trim()`. Aceeași verificare la `UpdateBook`, pe text tăiat.

- **M6 — a doua plasă din runda 10, neatinsă** (M5/r11): factory-urile nu verifică numărul de câmpuri, iar `UserTextMapper.cs:34` face `item.GetType()` pe `null`.

🟢 Cleanups

- **C1 — mesajele de validare.** Toate cele 9 mesaje noi din `Book` și `Course` încep cu spațiu sau cu bucata din câmp: `("Numele", " cartii trebuie sa...")` iese `Campul Numele nu e bun: cartii trebuie sa...` (două spații, „cartii" în regulă). Câmpul e `"Numele cartii"`, regula e `"trebuie sa aiba intre 2 si 30 de caractere"`. Restanțe din runda 11: `User.cs:37` `"uerului"`, `User.cs:132` câmpul `Userul` în loc de `Varsta`.
- **C2 — `Guid`-uri în mesajele pentru utilizator** (`BookService.cs:34`, `EnrolmentService.cs:50`): `Enrolmentul "id student d383e75c-... id curs 86737a36-..." e deja in lista.` Pentru utilizator ajung numele cărții, respectiv numele cursului.
- **C3 — la sfârșitul inputului, meniurile se învârt în gol.** Cu `while (true)`, `Console.ReadLine()` întoarce `null` la Ctrl+Z (Windows) / Ctrl+D, `TryParse` eșuează, iar bucla afișează meniul la nesfârșit: 2,6 milioane de linii în 4 secunde. `if (linie == null) return;` imediat după `ReadLine`.
- **C4 — `ViewAdmin.loggedUser`** tot nefolosit (C3/r11) · două rânduri goale rămase în `BookService.cs:30-31`.

Before / After

Unde	Acum	Cum ar trebui
`ViewStudent.cs:84`, `:113`	`case 0: break;`	`case 0: return;`
`ViewLogIn.cs:40`	`throw new ArgumentException("...");`	`throw new NotFoundException("Userul", lastName + " " + firstName);`
`ViewLogIn.cs:61`, `:75`	`throw new ArgumentException("Parola gresita!");`	`throw new ValidationException("Parola", "gresita");`
`ViewStudent.cs:199`, `:225`	`catch (DuplicateException ex)` `catch (NotFoundException ex)`	`catch (AcademyException ex)`
`ViewStudent.cs:267`	`enrolmentService.CreateEnrolment(...);`	`try { enrolmentService.CreateEnrolment(...); ... }` `catch (AcademyException ex) { ... }`
`Book.cs:38`	`ValidationException("Numele", " cartii trebuie...")`	`ValidationException("Numele cartii", "trebuie...")`

Q&A — în `RASPUNSURI.txt`, împreună cu cele din rundele 9, 10, 11 și din fișa Singleton

1. **B1:** în `Cursuri()` apeși `0`. Din ce iese `break`? De ce mergea în runda 11, când bucla era `while (tasta != 0)`, și de ce nu mai merge acum?

2. **B2:** desenează arborele de moștenire pentru `ArgumentException` și pentru `ValidationException`, până la `Exception`. Ce `catch` le prinde pe amândouă? De ce nu e o idee bună să-l pui pe acela în `Main`?
3. **B3:** `Main` prinde excepția și afișează mesajul corect. De ce se termină totuși programul? Pe ce linie continuă execuția după un `catch`?

▶ Ce urmează

Trei 🔴, toate trei pe același subiect: **unde se oprește execuția**, la `break`, la `throw` și la `catch`. Sunt fix-uri de o linie fiecare. Înainte de commit, rulează aceleași drumuri ca mine: `0` în fiecare meniu, un nume greșit la login, un nume de carte cu `!`. Apoi `RASPUNSURI.txt`, și timpul merge pe **lecția 2 de județeană**.

Runda 11 — 2026-10-07 — sha `e803498` — condensată

- ✅ **B1** `FindByStudentAndName` fără paranteze (method group) → **închis**, verificat prin rulare
- ⚠️ **B2** nume de carte invalid închidea aplicația → `Book` / `Course` convertite complet ✅, dar `catch`-urile din view tot înguste → **parțial** (B3/r12)
- ⚠️ **B3** înscriere dublă = stack trace → stack trace-ul a dispărut, dar sesiunea se termină → **parțial** (B3/r12)
- ⚠️ **B4** `continue` + `do-while(tasta != 0)` → ✅ în 3 meniuri · 🔴 în Cărți și Cursuri fix-ul a stricat ieșirea (B1/r12)
- ✅ **B5** `TryParse` ignorat la vârstă → **închis**
- 🟡 M1 `RASPUNSURI.txt` → încă (M1/r12) · M2 `jurnal.txt` în git → încă (M2/r12) · M3 calea jurnalului → parametru adăugat, nefolosit și greșit (M3/r12) · M4 confirmare la modificare → încă (M4/r12) · M5 a doua plasă → încă (M6/r12)
- 🟢 C1 mesaje → încă, extins la `Book` / `Course` (C1/r12) · C2 `Guid` în mesaj → încă (C2/r12) · C3 `loggedUser` → încă (C4/r12)
- 🔴 regresie nouă: schimbarea `catch`-ului din `Main` a lăsat login-ul fără plasă (B2/r12)
- Q&A → **fără răspuns**

Runda 10 — 2026-10-05 — sha `5f4704b` — condensată

- ✅ B2 `Update*` nu salvau → `Repository.Update` în 6 locuri, verificat după repornire
- ✅ B3 virgula în parolă → respinsă în `Admin` și `Teacher` · ⚠️ a doua plasă (număr de câmpuri, `ToText(null)`) nefăcută → M5/r11
- 🔴 B1 duplicat pe `Guid` nou → ✅ la cursuri și înscrieri · 🔴 **la cărți s-a stricat de tot** → B1/r11
- ✅ C1 · C2 · C3 · C5 · C6 (Strategy amânat, corect)
- 🟡 M1 `NotificatorConsola` fără drum → ✅ acum are drum (`ViewAdmin`) · M2 `jurnal.txt` în git → 🔴 încă (M2/r11) · M3 calea jurnalului → încă (M3/r11) · M4 stack trace la utilizator → ✅ `Main` afișează `.Message` · M5 `ModificareCarte` fără confirmare → încă (M4/r11)
- Q&A → **fără răspuns** (M1/r11)

Runda 9 — 2026-09-30 — sha `58681c9` — condensată

Observer funcționează (un `Add`, două reacții), zgomotul de la pornire a scăzut de la 29 de linii la 0, restanțele B3 și B4 din runda 7 au fost închise. Constatările:

- ✅ **B1** servicii construite de două ori în `ViewStudent` (inițializatori + constructor) → **închis** în runda 10.
- ✅ **B2** `ViewLogIn` abona un repository și pasa altul → **închis** (legătura). Notificatorul tot nu se aude, din alt motiv: M1, runda 10.
- ✅ **B3** `nameof(item)` scria literal „item" → **închis** (`typeof(T).Name`).
- ✅ **B4** `ThrowIfNull(nameof(interior))` nu declanșa niciodată → **închis**.
- ✅ **M1–M6** (`Repository<T> : INotificator`, `Display` mort, `typeof(UserTextMapper)`, `"\n"+Write`, typo-uri, newline final) → **toate închise**.
- ✅ **C1** vizibilități amestecate → **închis** · 🟡 **C2** calea jurnalului în 2 locuri → **încă deschis** (M3, runda 10).
- ⚠️ Q&A fără răspuns.

Runda 8 — 2026-09-29 — sha `067a141` — condensată

`TextMapperCuJurnal` = primul decorator adevărat al tău (aceeași interfață, referință la interior, deleagă întorcând rezultatul). Constatările:

- ✅ **B1** `nameof(T)` scria literal „T" → **închis** în runda 9 (`typeof(T).Name`).
- ✅ **B2** `new StreamWriter(cale)` trunchia, jurnal cu 1 intrare din 8 → **închis** (`, true`).
- ✅ **B3** ai schimbat toate cele 4 mappere deși fișa interzicea — dar te forțase B2 din runda 7 (`ITextMapper<T> : INotificator`); constrângerea era imposibil de respectat curat. **Închis** prin ruperea moștenirii.
- ✅ **B4** „repară linia, nu regula": `Console.WriteLine(Send(...))` scos doar la `UserTextMapper:72` → **închis**, toate cele 4 mappere curățate.
- 🟡 `return "";` inventat în `Send` → **încă prezent** în `Write` (M4, runda 9) · `Data/jurnal.txt` comis în git (fișier generat) → **încă în repo** · typo-uri → **M5** · 22 din 32 fișiere doar newline final normalizat de editor.
- **Restanțe runda 7:** B2 (Observer) ✅ închis în runda 9 · B3 (`DateTime.Parse` în Book) ✅ închis în runda 9, după 5 runde · B4 (`EnrolmentService` = `new()`) ✅ închis în `EnrolmentService`, 🔴 **reapărut de 3 ori în `ViewStudent`** (B1, runda 9).

Runda 7 — 2026-09-25 — sha `6bcc851`

Scop: verificat commit-ul `6bcc851` „factory pattern + decorator pattern + observer pattern" — adică tema din `EXERCITII-PATTERNS.pdf`. Metodă: `dotnet build` pe codul tău, apoi o copie în scratchpad pe care am **pornit efectiv aplicația** și am rulat drumul complet cerut de fișă (login → cărți → cursuri → statistici), plus testul de persistență și testul de 30 de secunde. Codul tău nu a fost modificat.

Poarta de compilare — **TRECE**

```
33 Warning(s)
0 Error(s)
```

Al treilea build verde consecutiv.

Bilanțul pe cele 6 cerințe din fișă

Cerință	Stare
0 — restanță: `BookTextMapper` să citească data ca `EnrolmentTextMapper`	🔴 **jumătate** — scrii corect, citești la fel ca înainte (B3)
1 — Factory Method peste `UserTextMapper.FromText`	✅ **FĂCUT, și făcut bine**
2 — Strategy / duplicarea din `Salary` și `Password`	🔴 **NEFĂCUT** — ai egalizat numerele, nu ai scos duplicarea (M3)
3 — Decorator peste `ITextMapper`	🔴 **NU EXISTĂ ÎN COD** (B1)
4 — Observer pe `Repository<T>`	🔴 **nu e Observer** (B2) — dar **rezultatul e atins**: se salvează pe disc
5 — provocarea, răspuns în scris	🔵 **nelivrat**

Ce merge — și e mai mult decât pare

Exercițiul 1 e făcut corect, în forma cerută. `IUserFactory` cu `Type` și `Create`, trei fabrici, array de fabrici, căutare după tip (`UserTextMapper.cs:65–72`). E fix forma din `ex10`, mutată pe cod adevărat. Nu am ce să-ți reproșez acolo.

Aplicația scrie acum pe disc — prima dată în șapte runde. Verificat prin rulare, nu prin citire:

```
adaugi „CarteDeTest" -> Repository: Item was added
inchizi, redeschizi, afisezi cartile:
   Nume carte: CarteDeTest , ID carte: 41a73ad9-... , adaugata in data: 9/25/2026
```

Și testul de 30 de secunde **trece**: pornire în care nu schimbi nimic → `Data/` neatins, toate patru fișierele cu același md5. Asta era „gata când"-ului exercițiului 4, și e îndeplinit. Mecanismul nu e Observer (vezi B2), dar problema pe care Observer-ul trebuia s-o rezolve — datele care dispăreau la închidere — **e rezolvată**.

`User` a devenit `abstract` (`User.cs:5`) — corect, nimeni nu mai poate instanția un utilizator fără tip. Drumul complet login → cărți → cursuri → statistici rulează fără să crape.

🔴 Critice

B1 — Decoratorul din mesajul commit-ului nu există în cod

`Common/IDisplay.cs` , `Common/ISaver.cs` , `Common/Repository.cs:3`

CUM E ACUM: commit-ul se numește „factory pattern + **decorator pattern** + observer pattern". `grep -rn "Decorator" --include="*.cs" .` întoarce **zero** rezultate. Nu există nicio clasă care implementează `ITextMapper<T>` și primește un `ITextMapper<T>` în constructor.

Ce există în schimb: două interfețe noi, `IDisplay` și `ISaver` , implementate **direct** de `Repository<T>` :

```
public class Repository<T> : IDisplay, ISaver where T : class, IEntity
```

DE CE asta nu e Decorator: un decorator implementează aceeași interfață ca lucrul pe care-l învelește **și ține o referință la el**. Cele două lucruri împreună sunt tot pattern-ul: ești interschimbabil cu originalul (aceeași interfață) și poți delega (referința). `Repository<T>` nu învelește nimic — și-a adăugat două metode proprii. E ca și cum ai fi spus „fac un ambalaj" și ai fi scris metoda direct în cutie.

Testul din fișă nici nu se poate rula: „îl pui pe `UserRepository` fără să schimbi `Repository<T>` și fără să schimbi vreun mapper". Nu există ce să pui — iar `Repository<T>` a fost schimbat.

FIX: clasa cerută, forma din `ex11` : implementează `ITextMapper<T>` , primește un `ITextMapper<T>` în constructor, scrie în consolă, apoi cheamă interiorul și întoarce ce a întors el. Îl montezi în `UserRepository` , la locul unde azi scrie `new UserTextMapper()` .

B2 — Notificarea pleacă la CITIRE, nu la scriere. Și nu e Observer.

`Users/Mappers/UserTextMapper.cs:69` · `Books/Mappers/BookTextMapper.cs:18` · `Courses/Mappers/CourseTextMapper.cs:18` · `Enrolments/Mappers/EnrolmentTextMapper.cs:19`

CUM E ACUM — reprodus prin rulare. Pornești aplicația și, **înainte** să apuci să te loghezi, primești 40 de linii de zgomot:

```
UserTextMapper: user type STUDENT created succesfully
UserTextMapper: user type STUDENT created succesfully
   ... de 8 ori, exact cate randuri are users.txt ...
==================LOG IN==================

Numele: Prenumele: CourseTextMapper Course Analizacreated succesfully
BookTextMapper: Book created Cartea1 succesfully
   ... inca 30 de linii, peste promptul in care tocmai scrii ...
```

Promptul „Numele:" și „Prenumele:" ajung îngropate în mesaje. Utilizatorul scrie peste ele.

DE CE se întâmplă: ai pus `Send(...)` în `FromText` . Dar `FromText` nu e chemat când se creează ceva — e chemat din `Repository.Read()` , o dată pentru fiecare linie din fișier, la construirea repository-ului. Mesajul spune „created succesfully" despre șapte cărți care există de săptămâni pe disc. Nu s-a creat nimic; s-a citit.

Momentul în care chiar se creează ceva e `Repository.Add()` — și de acolo nu pleacă nicio notificare către mapper.

Și de ce construcția nu e Observer: ai scris `ITextMapper<T> : INotificator` (`Common/ITextMapper.cs:3`), deci fiecare mapper **este** propriul lui notificator, cu `Console.WriteLine` scris înăuntru. Într-un Observer, sursa ține o **listă** de ascultători și îi anunță pe toți, fără să știe cine sunt. Aici nu există listă, nimeni nu se abonează, și nu poți adăuga al doilea ascultător fără să deschizi mapperul și să-i rescrii metoda `Send`.

Fișa cerea explicit **doi** ascultători, și spunea de ce: „Cu unul singur n-ai cum să vezi diferența dintre Observer și un simplu apel la `Save()` pus în `Add`." Exact asta ai acum — un apel direct, plus un `Console.WriteLine` legat de altceva.

FIX: sursa e `Repository<T>`. `Add` și `Remove` sunt momentele. Ține o listă de `INotificator` și anunț-o din ele. Ascultătorul care salvează și ascultătorul care scrie în consolă sunt două clase separate, niciuna în mapper.

B3 — Restanța din exercițiul 0, închisă pe jumătate — a patra oară

`Books/Mappers/BookTextMapper.cs:19`

CUM E ACUM:

```
// BookTextMapper — SCRIE corect:
item.CreatedAt.ToString("yyyy-MM-dd")
// BookTextMapper — CITESTE ca inainte:
DateTime.Parse(cuv[3])

// EnrolmentTextMapper — forma corecta, pe care o ai deja scrisa:
DateTime.ParseExact(cuv[3], "yyyy-MM-dd", CultureInfo.InvariantCulture)
```

„Gata când"-ul era: *cele două mappere citesc data în același fel*. `grep -n DateTime` pe cele două fișiere arată că nu.

DE CE contează, deși pare cosmetic: `Parse` fără format întreabă **cultura mașinii** cum să citească textul. `ToString("yyyy-MM-dd")` scrie întotdeauna la fel. Deci ai o pereche asimetrică: scrii ISO, citești „ce-o înțelege calculatorul ăsta, azi". Pe laptopul tău merge. Pe altă mașină, `2025-12-03` poate fi citit altfel sau deloc — și atunci nu primești o eroare clară, ci o dată greșită salvată înapoi în fișier la prima modificare.

Se vede deja în rulare, la afișarea cărților:

```
Nume carte: Cartea1 , adaugata in data: 12/3/2025 12:00:00 AM
```

+ același tipar, a doua jumătate: la adăugare formatezi data (`ViewStudent.cs:189`, `ToString("dd-MM-yyyy")`), la afișarea listei nu. Aceeași regulă, două locuri, ai reparat unul.

E a patra oară când repari o clasă și îi lași fratele neatins — ex3, ex7, restanța din fișă, și acum jumătatea de citire a aceleiași clase pe care tocmai ai atins-o.

B4 — `EnrolmentService` citește fișierul de două ori

`Enrolments/Services/EnrolmentService.cs:10`

CUM E ACUM — reprodus prin numărare. Am numărat mesajele de încărcare de la pornire și le-am comparat cu numărul de rânduri din fișiere:

```
  users.txt       8 randuri ->  8 mesaje
  courses.txt     6 randuri ->  6 mesaje
  books.txt       7 randuri ->  7 mesaje
  enrolments.txt  8 randuri -> 16 mesaje     <- de doua ori
```

DE CE se întâmplă:

```
private readonly EnrolmentRepository repository = new();   // (1) initializator de camp

public EnrolmentService(EnrolmentRepository repository)
{
    this.repository = repository;                          // (2) corpul constructorului
}
```

Inițializatorii de câmp rulează **înaintea** corpului constructorului. Deci (1) construiește un repository — care în constructorul lui citește tot fișierul — iar (2) îl aruncă imediat și pune altul, care citește fișierul a doua oară.

`readonly` nu te apără aici, și e bine să știi de ce: `readonly` permite atribuirea în **ambele** locuri, inițializator și constructor. Interzice doar scrierea de mai târziu, din alte metode. Compilatorul n-are ce să-ți semnaleze — ai scris ceva perfect legal.

Rezultatul e o a doua copie a înrolărilor în memorie, de care nu știe nimeni. Azi doar irosește o citire. Mâine, când pui Observer-ul din exercițiul 4 pe repository, salvarea poate pleca de la instanța orfană — și scrii pe disc o listă din care lipsesc modificările.

Celelalte trei servicii nu au tiparul — `BookService`, `CourseService` și `UserService` declară câmpul fără `= new()`. E singurul loc, deci probabil o rămășiță.

FIX: șterge `= new()` de pe linia 10.

🟡 Importante

- **M1** — `UserTextMapper.cs:18, :31, :44` — `AdminFromText`, `TeacherFromText`, `StudentFromText`: **zero apeluri** în tot proiectul. Fabricile le-au înlocuit, dar au rămas în fișier. Curăță-le odată cu exercițiul care le-a făcut inutile.
- **M2** — `UserTextMapper.cs:53` — `ToText` are încă lanțul de `if (item is X)`. E oglinda switch-ului pe care tocmai l-ai desființat la citire. Fișa cerea doar `FromText`, deci nu e o greșeală — dar întrebarea rămâne: când adaugi `PARINTE`, câte fișiere atingi acum?
- **M3** — exercițiul 2 nu e făcut. `grep -rn "minim pe economie" Users` întoarce **2** rezultate (`Teacher.cs:36`, `Admin.cs:33`); „gata când"-ul cerea unul. Ai schimbat pragul lui `Admin` din 3500 în 4257, deci acum cele două settere sunt copie identică. Hint-ul 1 spunea că una dintre perechi e „aceeași regulă cu număr diferit" și cealaltă „exact același cod" — tu ai mutat-o pe prima în a doua categorie, fără să scoți duplicarea.

- **M4** — `UserService.cs:123` — `Load()` returnează `new User[10]`, adică zece `null`-uri, și nu e chemată de nimeni.
- **M5** — `ViewLogIn.cs:21` — garda nouă crapă pe EOF: `Unhandled exception. System.NullReferenceException at ViewLogIn.Logger() line 21`. `Console.ReadLine()` întoarce `null` când intrarea se termină, iar tu ceri `.Length`. Pe Enter gol funcționează corect — problema e doar la închiderea intrării (Ctrl+Z pe Windows).
- **M6** — 33 de warning-uri, aproape toate `CS8600` / `CS8602` / `CS8604`: „possible null reference". M5 e fix unul dintre ele, devenit real. Merită citite, nu ascunse.

🟢 Cleanups

- **C1** — `CourseTextMapper.cs:18` — `"Course " + cuv[1] + "created succesfully"` → iese `Analizacreated`. Lipsește un spațiu.
- **C2** — formatele mesajelor diferă între mappere: `Name + ": "` la User și Book, `Name + " "` la Course și Enrolment.
- **C3** — `succesfully` → `successfully`, în toate patru.
- **C4** — `IDisplay` și `ISaver` sunt `internal`, `INotificator` e `public`. Alege una.
- **C5** — `Program.cs:18` — spațiu și indentare rămase după acolada de final.

Before / After

B4 — dublă citire

Acum	Corect
`private readonly EnrolmentRepository repository = new();`	`private readonly EnrolmentRepository repository;`
constructorul suprascrie, fișierul se citește de 2 ori	se citește o dată, cea injectată

B2 — unde se anunță

Acum	Corect
`FromText` cheamă `Send(...)` → anunț la fiecare linie citită de pe disc	`Add` / `Remove` anunță lista de ascultători
mapperul **este** notificatorul	mapperul nu știe că există ascultători
un singur ascultător, lipit în mapper	doi ascultători separați: unul salvează, unul scrie

B3 — citirea datei

Acum (`BookTextMapper`)	Corect (îl ai în `EnrolmentTextMapper`)
`DateTime.Parse(cuv[3])`	`DateTime.ParseExact(cuv[3], "yyyy-MM-dd", CultureInfo.InvariantCulture)`

Q&A — răspunde în scris

1. **B4:** de ce `readonly` nu a împiedicat ca acel câmp să fie scris de două ori? Ce anume interzice `readonly`, de fapt?
2. **B2:** ai un mapper care își scrie singur mesajul în consolă. Ce trebuie să atingi ca să adaugi un **al doilea** ascultător, care scrie în fișier? Și ce ar fi trebuit să atingi dacă era Observer?
3. **Restanța din fișă, exercițiul 1:** adaugi `PARINTE` cu un câmp în plus. Câte fișiere **existente** ești nevoit să modifici acum, cu fabricile? Câte erau cu `switch`-ul? Dacă răspunsul e același număr, spune-mi asta — e un rezultat corect.
4. **Exercițiul 5, rămas nelivrat:** după ce te loghezi și intri la cărți, câte copii ale lui `books.txt` ai în memorie? Ce se întâmplă cu Observer-ul din exercițiul 4 în situația asta?

Runda 6 — 2026-09-16 — sha `6839abf`

Scop: verificat commit-ul `6839abf` „fixes runda 5 + courses refactoring". Metodă: `dotnet build` pe codul tău, apoi o copie în scratchpad pe care am **pornit efectiv aplicația**, m-am logat, și am rulat un test de round-trip pe toate cele patru mapper-e (încarcă → salvează → compară fișierul). Codul tău nu a fost modificat.

Poarta de compilare — TRECE

```
35 Warning(s)
0 Error(s)
```

Al doilea build verde consecutiv. Regula a prins.

Runda cu cel mai bun bilanț de până acum: 5 din 7 critice închise

Runda 5	Stare azi
B1 — ai stricat `Books` cu prefixe în mapper fără prefix în fișier	✅ **ÎNCHIS**
B2 — `EnrolmentTextMapper` round-trip rupt din prima	✅ **ÎNCHIS**
B3 — `Save()` scrie `USER,` peste tot	⚠️ **jumătate** — `ToText` ramifică acum corect, dar efectul persistă din alt motiv (B1 de azi)
B4 — aplicația nu există, `Main` gol și `try` comentat	✅ **ÎNCHIS** — `Main` pornește `ViewLogIn`, `try`-ul e activ
B5 — din patru `Int32.Parse` ai șters trei	✅ **ÎNCHIS** — toate patru sunt `TryParse`
B6 — `UserTextMapper` nu ramifică pe tip (a patra oară)	🔴 **a CINCEA oară** — vezi B1
B7 — garda pe email doar la `CreateStudent`	✅ **ÎNCHIS** — e la toate trei

Ce s-a schimbat în felul în care lucrezi, și contează mai mult decât cifrele. B5 și B7 erau exact tiparul tău: repari linia pe care ți-o arăt, lași celelalte apariții ale aceleiași reguli. De data asta le-ai căutat singur pe toate. Al patrulea `Int32.Parse` de la `ViewStudent.cs:69` nu ți l-a mai arătat nimeni, și garda de email de la `CreateTeacher` / `CreateAdmin` nici atât. **Prima rundă în care ai aplicat regula, nu linia.**

`CourseRepository` e refactorizat corect, după același tipar ca `Books` și `Enrolments`, iar `CourseService` a scăzut de la 160 de linii la 40. Round-trip-ul pe `Books` și `Courses` e **OK**.

Ce a mers prost

Două lucruri, ambele găsite prin rulare, nu prin citire:

```
Books        round-trip: OK
Courses      round-trip: OK
Enrolments   round-trip: RUPT
Users        round-trip: RUPT
```

🔴 Critice

B1 — Nimeni nu se poate loga. A cincea rundă pentru același `if` lipsă

`Users/Mappers/UserTextMapper.cs:57–62`

CUM E ACUM: pornesc aplicația, introduc un utilizator care **există** în `users.txt`:

```
Numele: Scarlat
Prenumele: Stefan
System.ArgumentException: Error on user type
   at ViewLogIn.Logger() in ViewLogIn.cs:line 67
```

Aplicația te găsește, apoi nu știe ce ești. **Niciun login nu funcționează, deci nimic din aplicație nu e accesibil.** Și a doua față, demonstrată separat — o simplă încărcare urmată de salvare:

```
inainte: STUDENT,d383e75c-...,Stefan,Scarlat,stefanel.scarlat@gmail.com,18
dupa   : USER,d383e75c-...,Stefan,Scarlat,stefanel.scarlat@gmail.com,18
```

`users.txt` se corupe la prima salvare. Pentru un `Teacher` sau un `Admin` ar dispărea și salariul, orele și parola — câmpuri care nu există pe `User`, deci nu au unde să fie scrise.

DE CE: uită-te la cele două metode una lângă alta. `ToText` (linia 49) ramifică: `if (item is Student s) return StudentToText(s);`. Funcționează pentru că obiectul **există deja** și își poartă tipul cu el la execuție — `is` doar îl întreabă.

`FromText` nu are ce să întrebe. Ea **construiește** obiectul, deci în momentul apelului tipul încă nu există. Singurul loc unde mai e informația despre tip e chiar textul: `cuv[0]`, prefixul `STUDENT` / `TEACHER` / `ADMIN` pe care tu l-ai pus acolo. Linia 61 îl sare complet și construiește mereu `new User(...)`.

Asta e asimetria: **la scriere tipul e în obiect, la citire tipul e în date.** Dacă nu-l citești din date, se pierde definitiv — și tot ce urmează (`as Student` , `Save()`) lucrează cu un `User` gol.

Ai deja scrise `StudentFromText` , `TeacherFromText` , `AdminFromText` — liniile 14, 27, 40. **Toate trei sunt cod mort: nimeni nu le cheamă.** E al treilea tipar al tău, scrii piese pe care nu le conectezi, iar aici lipsește exact un `switch` pe `cuv[0]` .

Și un lucru care merită observat: bug-ul ăsta exista și în runda 5. Nu se vedea pentru că `Main` era gol. **Reparând B4, ai făcut vizibil B6.** Așa arată un proiect care începe să funcționeze — bug-urile ies la suprafață în ordinea în care ajungi la ele.

FIX: în `FromText` , ramifică pe `cuv[0]` și cheamă metodele pe care le ai deja.

B2 — Data se scrie altfel decât se citește. După o salvare, aplicația nu mai pornește

`Enrolments/Mappers/EnrolmentTextMapper.cs:10`

CUM E ACUM: încarc înscrierile și le salvez. Linia din fișier se transformă:

```
inainte: ...,db7a391a-ee78-4ae6-a3fb-a8c99a27fbf1,2025-10-20
dupa   : ...,db7a391a-ee78-4ae6-a3fb-a8c99a27fbf1,10/20/2025 12:00:00 AM
```

Apoi pornesc din nou aplicația, pe un calculator cu setări românești — adică al tău:

```
FormatException: String '10/20/2025 12:00:00 AM'
    was not recognized as a valid DateTime.
```

O salvare, și aplicația nu mai pornește niciodată.

DE CE: `item.CreatedAt` fără format cheamă `ToString()` implicit, iar acela folosește **cultura calculatorului pe care rulezi**. Pe o mașină americană iese `10/20/2025` , pe una românească `20.10.2025` . `DateTime.Parse` la citire face simetric: interpretează după aceeași setare. Deci formatul fișierului tău nu e o decizie a ta — e o consecință a setărilor din Windows.

De aici vine `FormatException` : șirul `10/20/2025` citit cu reguli românești cere **luna 20**, care nu există.

Compară cu geamănul lui, `Books/Mappers/BookTextMapper.cs:10` :

```
item.CreatedAt.ToString("yyyy-MM-dd")
```

Format fix, identic pe orice calculator. **L-ai reparat pe acesta chiar în runda asta** — și l-ai lăsat neatins pe celălalt, deși ai modificat `EnrolmentTextMapper` în același commit. Cele două fișiere au 19 linii fiecare și sunt aproape identice.

Ăsta e tiparul „repară linia, nu regula” în forma lui cea mai curată. Ai închis B5 și B7 căutând toate aparițiile — aici nu ai căutat.

FIX: `.ToString("yyyy-MM-dd")` la scriere. La citire, `DateTime.ParseExact(cuv[3], "yyyy-MM-dd", CultureInfo.InvariantCulture)` — ca să nu mai depindă de setări nici la intrare.

Before / After

B1 — `UserTextMapper.cs:57`

Acum:

```
public User FromText(string text)
{
    string[] cuv = text.Split(',');

    return new User(Guid.Parse(cuv[1]), cuv[2], cuv[3], cuv[4],
                    Int32.Parse(cuv[5]));
}
```

Cum ar trebui:

```
public User FromText(string text)
{
    string[] cuv = text.Split(',');

    switch (cuv[0])
    {
        case "STUDENT": return StudentFromText(text);
        case "TEACHER": return TeacherFromText(text);
        case "ADMIN":   return AdminFromText(text);
        default:
            throw new ArgumentException(
                "Tip de user necunoscut: " + cuv[0]);
    }
}
```

B2 — `EnrolmentTextMapper.cs:10` și `:17`

Acum:

```
return item.Id + "," + item.StudentId + ","
       + item.CourseId + "," + item.CreatedAt;

return new Enrolment(..., DateTime.Parse(cuv[3]));
```

Cum ar trebui:

```
return item.Id + "," + item.StudentId + "," + item.CourseId + ","
       + item.CreatedAt.ToString("yyyy-MM-dd");

return new Enrolment(..., DateTime.ParseExact(cuv[3], "yyyy-MM-dd",
                          CultureInfo.InvariantCulture));
```

Ordinea de lucru

1. **B1** — `switch` pe `cuv[0]`. E `if`-ul care îți deblochează toată aplicația.
2. **Pornește și loghează-te.** Abia acum poți vedea ecranul studentului, pe care l-ai scris în august și nu l-ai văzut niciodată rulând.
3. **B2** — formatul datei, în ambele sensuri.
4. **Rulează testul de round-trip pe toate patru** înainte de commit.

Q&A

1. `ToText` ramifică pe tip și funcționează. `FromText` are aceeași structură de date în față și nu poate face la fel. **Ce are una și nu are cealaltă, în momentul apelului?**
2. Ai avut build verde și în runda 5, și în runda 6. În amândouă, aplicația era inutilizabilă din motive diferite. **Ce test, rulat în 30 de secunde, ar fi prins ambele situații?**
3. Ai reparat data în `BookTextMapper` și ai lăsat-o în `EnrolmentTextMapper`, în același commit. La `Int32.Parse` și la garda de email ai făcut invers — le-ai căutat pe toate. **Ce a fost diferit? Cum ți-ai dat seama acolo că e o regulă, și aici nu?**

Runda 5 — 2026-09-15 — sha `eb5c7a8`

Primul build verde din nouă commit-uri. 7 constatări critice; starea fiecăreia e tabelată în runda 6, mai sus. Pe scurt: B1 (prefixe în `Books`) ✅ · B2 (round-trip `Enrolment`) ✅ · B3 (`Save()` scrie `USER,`) ⚠️ jumătate · B4 (aplicația nu pornea) ✅ · B5 (`Int32.Parse`) ✅ · B6 (`UserTextMapper` nu ramifică pe tip) 🔴 a cincea oară, închisă abia în `deab1cf` · B7 (garda pe email doar la `CreateStudent`) ✅.

Runda 4 — 2026-09-15 — sha `aba4e01`

Verificat commit-ul „code review fixes”. Buildul a picat a șasea oară — **1 eroare vizibilă, 6 ascunse în spatele ei**, pentru că `Enrolment` nu implementa `IEntity` și legarea tipului de bază oprea verificarea corpurilor. Prima rundă cu bilanț pozitiv pe restanțe (3 🔴 + 5 🟡 închise), dar aplicația a fost **golită** ca să treacă buildul.

#	Constatare	Stare azi
B1	nu compilează: `Enrolment` fără `IEntity` + 6 erori ascunse (apelanți neactualizați după mutare)	✅ **ÎNCHIS** — build verde, toți apelanții updatați
B2	aplicația nu mai există: `Main` gol, `Logger` comentat integral	🔴 NEATINS (R5 B4)
B3	`TryParse` lipit **sub** `Int32.Parse`, ×4 → meniul stricat și pentru input corect	⚠️ trei sferturi — `:69` rămas (R5 B5)
B4	`users.txt` a trecut la prefixe, `UserTextMapper` nu → contradicție garantată	⚠️ jumătate — fișierul se citește ✅, dar tot iese `User` (R5 B6)
B5	`Save()` rade salarii/parole și șterge prefixele	🔴 **înrăutățit** (R5 B3) — scrie `USER,` peste toate tipurile
B6	garda pe email doar la `CreateStudent`	🔴 NEATINS (R5 B7)
B7	`EnrolmentService` cu două liste paralele; `repository` nefolosit	✅ **ÎNCHIS**
M1	`EnrolmentRepository.FindById` ascunde metoda moștenită	🔴 NEATINS (R5 M2)
M2	`EnrolmentRepository` fără constructor `base(...)`	✅ ÎNCHIS
M3	`EnrolmentMapper` / `EnrolmentTextMapper` clase goale	✅ ÎNCHIS (dar round-trip rupt — R5 B2)
M4	`Email`: lungimea verificată înaintea lui `Trim`	🔴 NEATINS
M5	`DeleteUser` nu verifică dacă userul există	🔴 NEATINS
M6	nimic nu cheamă `Save()` pentru useri	🔴 **înrăutățit** — și `EnrolmentService` și-a pierdut `Save()`
M7	`InscriereCurs` ocolește serviciul	✅ ÎNCHIS
M8–M12	confirmare lipsă la `UpdateBook`, login tăcut, servicii auto-instanțiate, cod mort, naming DTO	⚠️ M10 jumătate; restul NEATINSE
C1–C8	`.gitignore`, cod comentat, `students.txt`, `aux`, parole, warnings, linie goală, newline la final	✅ C2 (`User.cs`); restul NEATINSE

Mecanismele explicate atunci și încă valabile: **o eroare de legare a unui tip suprimă erorile din tot codul care îl folosește** (B1) · `/* */` scoate codul din build, deci „build verde" devine un semnal fals (B2) · `Console.ReadLine()` **consumă** o linie la fiecare apel, deci două apeluri cer două linii (B3) · **tipul static al parametrului decide ce câmpuri sunt vizibile**, de-aia `ToText(User)` nu poate scrie un salariu (B5) · o constatare are un *unde* și o *regulă*; adresa e doar punctul din care ți-am arătat regula (B6).

Runda 3 — 2026-09-09 — sha `70360d6`

Verificat commit-ul „more user fixes". Buildul a picat cu 4 erori. Partea de fond s-a mișcat serios (B4 din R2 închis pe bune), dar `Student` / `Teacher` / `Admin` au primit `ITextMapper<T>` — șase metode pe care nimeni nu le putea chema.

#	Constatare	Stare azi
B1	nu compilează: `ViewLogIn:10`, `:21`, `Teacher.cs:87` — apelanți neactualizați după mutare	🔴 redeschis (R4 B1), același tipar
B2	aplicația nu pornește; după reparare, nici studenții nu se pot loga	🔴 NEATINS (R4 B4)
B3	`ITextMapper<Student>` nu e `ITextMapper<User>` → cele 6 metode noi, necheamabile; `Save()` rade parole	🔴 NEATINS (R4 B4, B5)
B4	fiecare `FromText` contrazice `ToText`-ul de deasupra (prefix + câmpuri lipsă)	✅ **ÎNCHIS** — round-trip verde pe toate trei
B5	`Admin` : `salary` înaintea lui `age` → admin de 10000 de ani cu salariul 48	✅ **ÎNCHIS**
B6	`if (FindById(student.Id) != null)` — mereu falsă, `Id` proaspăt generat	⚠️ o treime — `CreateStudent` ✅, `CreateTeacher` / `CreateAdmin` 🔴 (R4 B6)
B7	`Int32.Parse` pe tastă, 4 locuri	🔴 **înrăutățit** (R4 B3) — `TryParse` lipit sub `Parse`
B8	`Console.WriteLine` în loc de `throw` la `Salary` / `Password`	✅ **ÎNCHIS**, toate patru
M1	`UserRepository.FindById` ascunde metoda moștenită	✅ ÎNCHIS — refăcut în `EnrolmentRepository` (R4 M1)
M2	`Email` nu se curăță de spații	⚠️ jumătate — `email = text` ✅, lungimea tot pre-`Trim` (R4 M4)
M3	`UserMapper` nu e `static`	✅ ÎNCHIS
M4	`ToStudentUpdateResponse(User)` față de frații ei	✅ ÎNCHIS
M5	nimic nu cheamă `Save()` pentru useri	🔴 NEATINS (R4 M6)
M6	`DeleteUser` nu verifică dacă userul există	🔴 NEATINS (R4 M5)
M7	`StudentCreateRequest` în alt namespace	✅ ÎNCHIS

#	Constatare	Stare azi
M8–M12	cod mort, `InscriereCurs`, confirmare lipsă, login tăcut, servicii duble	🔴 NEATINSE (R4 M7–M11)
C1–C9	cod comentat, `.gitignore`, `students.txt`, `aux`, parole, warnings, `using`-uri, newline, naming	✅ C7; C1 🔴 înrăutățit (+62 de linii); restul NEATINSE

Fix-urile complete, cu cod, pentru fiecare dintre cele de mai sus sunt în runda 4 (pentru cele încă deschise) — iar mecanismele explicate atunci rămân valabile: invarianța generică la B3, round-trip-ul la B4, „`int` e `int`" la B5, identitate vs. date la B6.

Runda 2 — 2026-09-09 — sha `3c1877d`

Verificat pasul 3 din Faza 1 (`User` → T0.5). Buildul a picat cu 16 erori (6 vizibile, 10 ascunse în spatele lor). Pașii 1 și 2 — `Course` și `Enrolment` — sări.

#	Constatare	Stare azi
B1	nu compilează: metode duplicate în `UserMapper` + DTO-uri șterse fără apelanți	🔴 redeschis (R3 B1)
B2	`UserTextMapper.FromText` face `Guid.Parse` pe prefixul de tip → aplicația nu pornește	🔴 NEATINS (R3 B2)
B3	`FromText` întoarce mereu `User` → adminul devine student, `Save()` rade parola și salariul	🔴 NEATINS (R3 B2, B3)
B4	`CreateUser` : `if (request != null)` mereu adevărat → orice cerere produce un `Student`	✅ ÎNCHIS
B5	`Salary` / `Password` în `Teacher` și `Admin` : `Console.WriteLine` în loc de `throw`	🔴 NEATINS (R3 B8)
B6	`Int32.Parse` pe tastă în `ViewStudent`	🔴 NEATINS (R3 B7)
M1	`UserRepository.FindById` ascunde metoda moștenită (`CS0108`)	🔴 NEATINS
M2	`Email` nu se face `Trim` , deși `FirstName` / `LastName` da	🔴 NEATINS
M3	`UserMapper` nu e `static` ; `ToTeacher` întoarce `User`	⚠️ jumătate
M4	ordinea argumentelor: `Teacher` greșit, `Admin` — capcană anunțată	⚠️ `Teacher` ✅ / `Admin` 🔴 (R3 B5)
M5	`StudentCreateRequest` în alt namespace decât frații lui	🔴 NEATINS
M6–M10	cod mort, `InscriereCurs` ocolește serviciul, modificarea cărții nu confirmă, login admin/profesor tăcut, servicii instanțiate de mai multe ori	🔴 NEATINS
C1–C7	cod comentat, `.gitignore` , `students.txt` mort, `Console.WriteLine(aux)` , parole în clar, warnings, `using` -uri	⚠️ C1 parțial; restul NEATINSE

Runda 1 — 2026-09-08 — sha `79abec3`

Verificat T0.0–T0.4 din `TASKS.md`. Buildul a trecut. **5/5 task-uri închise pe fond.**

#	Constatare	Stare azi
T0.0–T0.4	nu compilează · „Vezi cursurile" crapă · modificarea orfanizează cartea · `createdAt` ignorat · `CreateUser` face mereu un `User` gol	✅ toate 5 închise
B1	`Contains` la căutarea după nume → Enter pe gol înseamnă „primul din listă"	✅ ÎNCHIS
B2	`null` strecurat în lista de cursuri	✅ ÎNCHIS
B3	profesorul salvat nu mai poate fi citit (`ToText` ≠ constructorul din text)	🔴 reapărut de 3 ori în modele (R3 B4)
B4	`Int32.Parse` pe tastă	🔴 NEATINS de 2 runde (R3 B7)
M1	`Create` / `Update` din modele = cod mort	✅ șterse
M2	`BookUpdateRequest` cere o dată ignorată	✅ ÎNCHIS de T0.5
M3	`GetEnrolmentIdByStudentId` cod mort	🔴 NEATINS
M4	modificarea cărții nu confirmă nimic	🔴 NEATINS
M5	`User newUser = new();` inutil	✅ ÎNCHIS
M6	`InscriereCurs` ocolește serviciul	🔴 NEATINS
M7	login admin/profesor iese tăcut	🔴 NEATINS
M8	serviciile se instanțiază de mai multe ori	⚠️ `BookService` ✅, restul 🔴
C1–C7	`Console.WriteLine(aux)` · `.gitignore` · `students.txt` · parole în clar · warnings · `override` gol · `using` -uri	✅ C6; restul 🔴